
**Institute of Engineering and Technology (IET)**

JK Lakshmipat University, Jaipur

# Minor Project Final Report

# Mess Management Portal

**PREPARED BY**

Aman Singh (2023BTech006)

Anjisht Amritanshu (2023BTech009)
Ayush Choudhary (2023BTECH020)

**FACULTY GUIDE**

Dr. Deepika Prakash
Assistant Professor | Computer Science and Engineering

April 2026

# Contents

# List of Tables

# 1 Abstract

The JKLU Mess Management Portal is a comprehensive full-stack web application designed to digitize and streamline the cafeteria (mess) entry and meal management system at JK Lakshmipat University. The existing physical ID card-based entry system was prone to inefficiencies such as long queues, proxy entries, and lack of real-time tracking and meal wastage data. This project replaces the traditional system with a QR code-based digital mess pass, enabling students to carry a digital identity, mess staff to scan and verify entries in real time, and administrators to monitor headcount, manage menus, track meal cancellations, and issue notices.

The portal is built using the MERN stack (MongoDB, Express.js, React, Node.js), with Microsoft Azure AD for authentication via institutional email IDs. The system supports three user roles — Hostel students, Day Scholar students and mess staff (administrators) — each with tailored interfaces and role-based access control (RBAC).

Key features include QR-based meal verification with time-based meal detection, a meal skip/booking system with monthly quotas and automated rebate calculation, real-time kitchen headcount analytics, a refund ledger with CSV export, per-dish nutritional information with an expandable database of over 100 dishes, Razorpay-integrated non-vegetarian meal booking with HMAC-SHA256 payment verification, dietary preference filtering (Vegetarian, Non-Vegetarian, Eggetarian, and Jain Food), a Reddit-style community issue polling and resolution system with upvote/downvote mechanics and trending algorithms, food item exclusion for advance meal customisation, a meal pricing display engine, a complaint module with photographic evidence upload, a review and star-rating system for each meal, and a premium glassmorphism UI with blur-modal interactions and micro-animations.

The final deliverable encompasses eleven Mongoose database models, six backend controllers, eight RESTful API route groups, nine frontend page components, and four reusable UI components — constituting a production-ready, scalable web application.

# 2 Introduction

University mess management is a critical aspect of campus life that directly impacts student satisfaction and institutional operational efficiency. At JK Lakshmipat University (JKLU), the mess serves hundreds of students daily across four meal slots — Breakfast, Lunch, Snacks, and Dinner. The current system relies on physical ID cards for entry verification, manual headcount tracking, and paper-based or informal feedback collection.

This project, the **Mess Management Portal**, aims to modernize this process by building a comprehensive web-based platform that serves three key stakeholders:

1. **Students**: Receive a digital QR code-based mess pass, view daily and weekly menus, skip or book meals with a monthly quota system, view meal history, pay for additional dishes and submit food reviews with photo evidence.

2. **Mess Staff**: Use a built-in QR scanner to verify student entries in real time, with automatic duplicate detection and time-based meal-type resolution.

3. **Administrators**: Manage weekly menus, view tomorrow's headcount for kitchen planning, generate refund ledgers for cancelled meals, broadcast notices, and review student feedback.

The portal leverages Microsoft Azure Active Directory single sign-on (SSO) for authentication, ensuring that only students with valid `@jklu.edu.in` email addresses can access the system. The application is built on the MERN stack, combining a React frontend (bootstrapped with Vite) with a Node.js/Express backend and MongoDB Atlas as the cloud database. Additionally, the system integrates Razorpay as a payment gateway for non-vegetarian meal bookings and implements a comprehensive nutritional information database for dietary awareness.

This report documents the complete project, including system architecture, database design, API development, frontend implementation, testing, and the final feature set delivered.

# 3 Problem Statement

The existing mess management system at JKLU suffers from several operational and administrative challenges:

1. **Inefficient Entry Verification**: Physical ID cards are manually checked at the mess entrance, leading to long queues during peak meal hours and the possibility of proxy entries.

2. **No Real-Time Tracking**: There is no digital record of which students have eaten which meals on a given day. This makes it impossible to generate accurate meal consumption reports or detect anomalies.

3. **Food Wastage**: Without advance knowledge of how many students plan to eat, the kitchen prepares food based on rough estimates, often resulting in significant food wastage or shortages.

4. **Lack of Meal Flexibility**: Students have no mechanism to skip a meal in advance or track how many meals they have opted out of in a month, hindering any potential refund or credit system.

5. **No Feedback Channel**: There is no structured way for students to rate meals, report food quality issues, or attach photographic evidence of problems, making it difficult for administrators to identify and address recurring issues.

6. **Administrative Overhead**: Menu updates, headcount tracking, refund calculations, and notice distribution are handled manually or through informal channels, consuming valuable administrative time and introducing errors.

The Mess Management Portal addresses all of these problems through a unified digital platform that automates entry verification via QR codes, provides real-time meal logging, supports advance meal booking and cancellation with enforced quotas, offers kitchen headcount analytics, and includes a comprehensive review and feedback system.

# 4 Methodology and Theoretical Background

## 4.1 Development Methodology

The project follows an **Agile development methodology** with iterative sprints. The development is divided into the following phases:

1. **Requirements Gathering and Analysis**: Identifying stakeholder needs through discussions with mess administration and students.

2. **System Design**: Designing the database schema, API architecture, authentication flow, and UI wireframes.

3. **Implementation**: Incremental development of backend APIs first, followed by frontend integration.

4. **Testing**: Unit testing of individual API endpoints, integration testing of the full authentication and scanning flow.

5. **Deployment and Feedback**: Deploying the application and gathering user feedback for iterative improvement.

## 4.2 Theoretical Background

### 4.2.1 QR Code-Based Authentication

QR (Quick Response) codes are two-dimensional barcodes capable of encoding arbitrary data. In this project, each student's unique MongoDB ObjectId is encoded into a QR code, which serves as their digital mess pass. The QR code is generated client-side using an external API (`api.qrserver.com`) and is displayed on the student's dashboard. Mess

staff scan this QR code using the device camera via the `@yudiel/react-qr-scanner` library, and the decoded student ID is sent to the backend for verification.

### 4.2.2 OAuth 2.0 and Microsoft Azure AD SSO

The system uses Microsoft Azure Active Directory (Azure AD) for authentication via the OAuth 2.0 authorization code flow. The Microsoft Authentication Library (MSAL) for JavaScript handles the popup-based login flow on the frontend. Only email addresses matching the `@jklu.edu.in` domain are accepted, enforced on both the frontend and backend. Upon successful authentication, the user's profile is upserted into MongoDB, and the session is maintained via `localStorage`.

### 4.2.3 RESTful API Design

The backend follows REST (Representational State Transfer) principles. Resources (users, meals, bookings, menus, nutrition records, complaints, polls, and notices) are exposed through clearly defined HTTP endpoints organised by domain (`/api/auth`, `/api/mess`, `/api/dashboard`, `/api/admin`, `/api/complaints`, `/api/nutrition`, `/api/payment`, `/api/polls`). Standard HTTP methods (GET, POST, PUT, DELETE) are used for CRUD operations, and responses follow a consistent JSON format (`{success, data, message}`).

### 4.2.4 Time-Based Meal Detection

Instead of requiring the mess staff to manually select a meal type during scanning, the system automatically determines the current meal based on the server clock:

- 08:00 – 09:30 → Breakfast
- 12:00 – 14:00 → Lunch
- 17:00 – 18:00 → Snacks
- 20:00 – 22:00 → Dinner
- Outside these windows → Mess is closed

This eliminates manual selection errors and ensures accurate meal logging.

### 4.2.5 Monthly Quota-Based Meal Skip System

Students can skip meals in advance (for the next day). To prevent abuse, the system enforces monthly skip quotas per meal type:

- Breakfast: 5 skips/month

- Lunch: 5 skips/month
- Snacks: 10 skips/month
- Dinner: 5 skips/month

Quota enforcement is performed server-side to prevent client-side manipulation. The remaining quota is displayed on the student dashboard. Meals that are skipped contribute to a monthly rebate ledger, which is accessible by the accountant role for processing refunds.

#### 4.2.6 Razorpay Payment Gateway Integration

For non-vegetarian meals that carry an additional charge and meal charge for day scholar students, the system integrates the Razorpay payment gateway. The payment flow follows a three-step process:

1. **Order Creation**: The backend creates a Razorpay order with the amount in paise (smallest currency unit) and returns the order ID to the frontend.
2. **Checkout**: The frontend dynamically loads the Razorpay checkout script and opens the payment modal with pre-filled student details.
3. **Verification**: Upon payment completion, the backend verifies the payment authenticity using **HMAC-SHA256 signature verification** — computing a hash of `orderId|paymentId` using the secret key and comparing it against Razorpay's signature.

This ensures that payment confirmations cannot be forged by malicious clients.

#### 4.2.7 Nutritional Information System

The system maintains a dedicated `Nutrition` collection containing per-dish nutritional data (calories, protein, carbohydrates, fat, dietary fibre, and free sugar) for over 100 commonly served mess dishes. Nutritional data is displayed alongside each dish in expandable panels with animated progress bars. The admin interface includes a **fuzzy search** engine with a custom scoring algorithm that ranks results by substring match quality, enabling efficient dish lookup even with partial or misspelled queries.

#### 4.2.8 Community Issue Polling System

The polling module implements a Reddit-style post system where hostellers can raise mess-related issues (food quality, hygiene, service, timing, cleanliness). Each post supports upvote/downvote mechanics using MongoDB's `$addToSet` and `$pull` atomic operators. Posts can be sorted by three algorithms:

- **Recent**: Chronological order by creation timestamp.
- **Top**: Net vote score (upvotes minus downvotes).
- **Trending**: A decay-weighted score computed as $\frac{\text{net votes}}{(\text{age in hours}+2)^{1.5}}$, which surfaces recently popular posts.

Resolution follows a two-step workflow: the admin marks the issue as fixed, then the original poster confirms the resolution, after which the post is archived.

#### 4.2.9 Dietary Preference Filtering

Students can configure their dietary preference (Vegetarian, Non-Vegetarian, Eggetarian, or Strict-Vegetarian/Jain Food) in their profile settings. The menu pages apply client-side filtering to hide or show items accordingly:

- **Vegetarian / Jain**: All non-veg items are hidden.
- **Eggetarian**: Only egg-based items (matched via keyword detection) are shown from the non-veg section.
- **Non-Vegetarian**: All items are visible.

#### 4.2.10 Meal Pricing and Rebate System

Each meal type carries a predefined cost displayed on the student dashboard. Non-vegetarian items have tiered pricing: egg-based items at Rs.30 and chicken-based items at Rs.120 per serving, for now. The rebate system automatically tracks all cancelled meals per student per month. The refund ledger, accessible to accountants, aggregates these cancellations and cross-references them with user identity data, enabling accurate monthly refund processing with CSV export capability.

#### 4.2.11 Food Item Exclusion

Students can exclude specific unwanted food items from their meal in advance. When viewing tomorrow's menu, students can deselect individual items they do not wish to receive. This information is stored in the `MealBooking` collection and relayed to the kitchen via the admin headcount analytics dashboard, enabling more precise food preparation quantities and reducing wastage.

#### 4.2.12 Review and Rating System

Students can submit reviews and star ratings (1–5 scale) for each meal they have consumed. Reviews can include text comments and photographic evidence uploaded via the

`FormData` API and processed by Multer middleware. The admin and contractor roles can view all reviews in a filterable grid, enabling data-driven improvements to food quality and service.

#### 4.2.13 Glassmorphism UI with Blur-Modal Interactions

The frontend employs modern UI patterns including glassmorphism (frosted-glass effects using CSS `backdrop-filter: blur()`), gradient-based colour-coded meal tiles, spring-physics animations (`cubic-bezier(.34,1.56,.64,1)`), and micro-interactions (hover lifts, scale transforms, pulsing indicators). Modals use a blurred backdrop overlay with slide-up entry animations, creating a premium, app-like user experience.

# 5 Architecture and Key Design Decisions

## 5.1 High-Level System Design

The application follows a standard **three-tier architecture**:

- **Presentation Layer (Frontend)**: A React application bootstrapped with Vite, communicating with the backend via Axios HTTP calls. Uses MSAL.js for Microsoft Azure AD SSO and vanilla CSS3 (glassmorphism, CSS Grid, Flexbox) for styling.
- **Application Layer (Backend)**: A Node.js server using Express 5 framework. Handles authentication, business logic (meal scanning, booking, quota enforcement), and data access through Mongoose ODM.
- **Data Layer (Database)**: MongoDB Atlas (cloud-hosted NoSQL database) stores all application data — users, meal logs, bookings, menus, complaints, nutrition records, poll posts, and notices.

## 5.2 Tech Stack

## 5.3 Additional Design Decisions

1. **Domain-Restricted Authentication**: Both frontend and backend validate that the user's email ends with `@jklu.edu.in`. The backend returns HTTP 403 for non-university emails, preventing unauthorized access.
2. **Upsert Pattern for User Management**: The `microsoftLogin` controller uses a find-or-create (upsert) pattern. If a student logs in for the first time, a new user record is created; subsequent logins update the existing record. This eliminates the need for a separate registration flow.

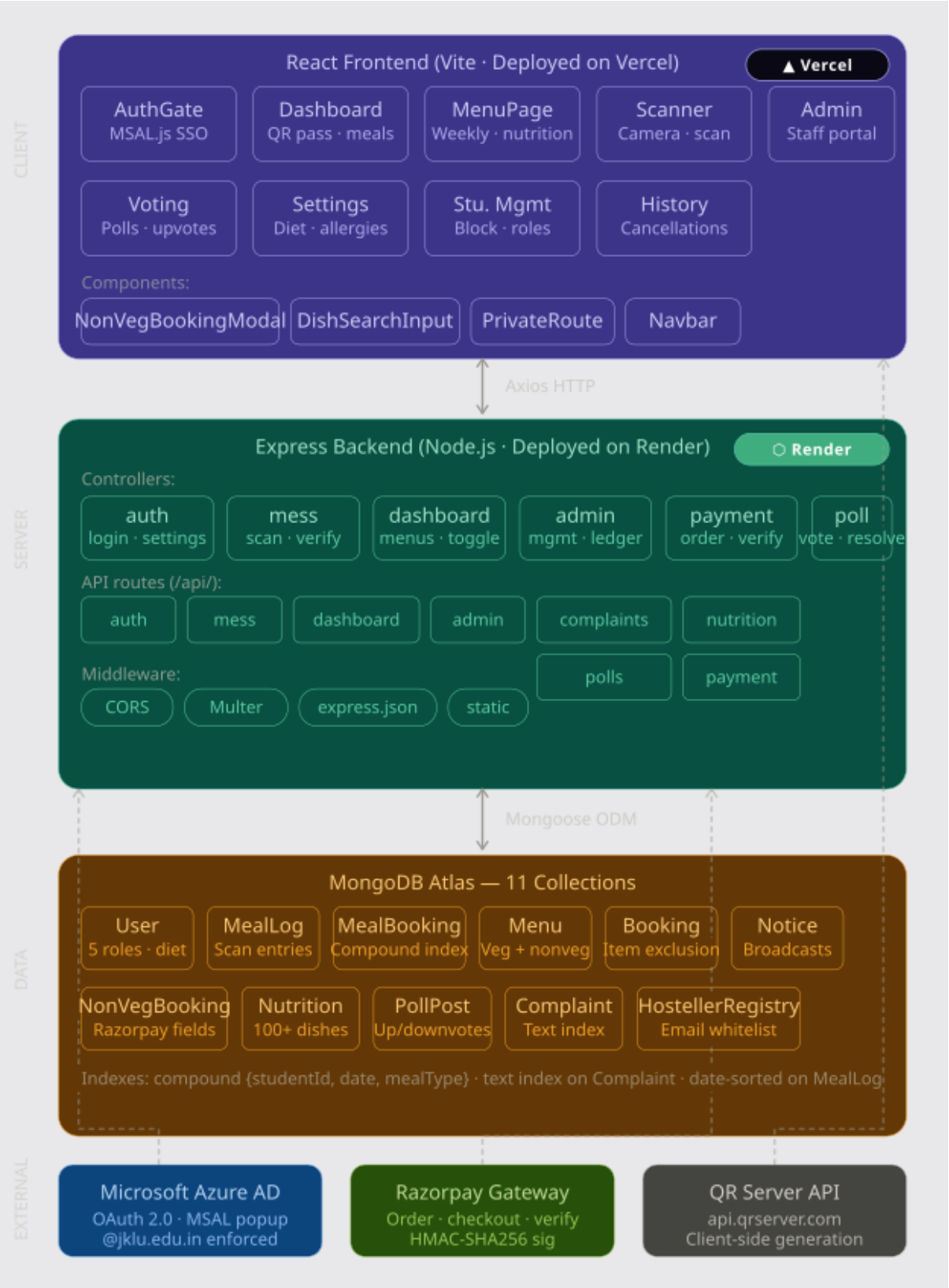


Figure 1: Architecture

| Layer | Technologies |
|---|---|
| Frontend | React 18 (Vite), React Router v6, MSAL.js, Axios, @yudiel/react-qr-scanner, lucide-react, CSS3 (Glassmorphism, backdrop-filter, CSS Grid, Flexbox) |
| Backend | Node.js, Express 5, Mongoose ODM, CORS, Multer (file uploads), dotenv, crypto (HMAC-SHA256), Razorpay SDK |
| Database | MongoDB Atlas (cloud-hosted NoSQL) — 11 collections with compound indexes |
| Authentication | Microsoft Azure Active Directory (OAuth 2.0 popup flow via MSAL.js) |
| Payment Gateway | Razorpay (order creation, checkout, HMAC-SHA256 signature verification) |
| Hosting/DevOps | Vite dev server (frontend), Node.js (backend), MongoDB Atlas (database) |

Table 1: Technology Stack Summary

3. **Hosteller Registry Whitelist**: A dedicated `HostellerRegistry` collection acts as the source of truth for hostel residency. On every login, the system cross-references the student's email against this registry and automatically sets their `residencyStatus` to Hosteller or Day-Scholar. This ensures residency changes made by the admin take immediate effect without requiring students to update their profile.

4. **Dual Meal Tracking Models**: The system maintains two separate collections:

   - `MealLog` — Records real-time scan entries (what students actually ate).
   - `MealBooking` — Records advance booking/cancellation decisions (what students plan to eat tomorrow).

   This separation ensures that planned and actual consumption data remain independently queryable.

5. **Compound Unique Index on MealBooking**: The `MealBooking` collection has a unique compound index on `{studentId, date, mealType}`, preventing duplicate booking entries at the database level.

6. **HMAC-SHA256 Payment Verification**: Non-vegetarian meal payments are verified using cryptographic signature verification. The backend computes `HMAC-SHA256(orderId|secret)` and compares it against Razorpay's signature, ensuring payment integrity.

7. **Client-Side QR Generation**: QR codes are generated using an external API (`api.qrserver.com`) rather than server-side rendering. This reduces backend load and simplifies the architecture, as the student's MongoDB `_id` is a stable, unique identifier.

8. **Role-Based Access Control (RBAC)**: The system supports five distinct roles — `student`, `admin`, `contractor`, `accountant`, and `controller` — each with tailored UI rendering:
    - `admin`, `contractor` → Kitchen Controls, Menu Management, Reviews, and Poll Resolution
    - `admin`, `accountant` → Refund Ledger and Financial Analytics
    - `admin` → Student Management, Hosteller Registry, Role Assignment, Notice Broadcast
9. **Fuzzy Search with Custom Scoring**: The nutrition API implements a custom fuzzy matching algorithm that scores candidates based on character-sequence matching. Substring matches receive maximum score; partial matches are scored proportionally. Results are capped at 10 for performance.
10. **Debounced API Calls**: All search inputs across the application (dish search, poll search) implement 280–300ms debouncing to minimise unnecessary API calls during rapid typing.
11. **Static File Serving for Uploads**: Complaint and review images uploaded via Multer are stored in the `uploads/` directory, which is served as a static directory by Express. This avoids the complexity of a dedicated file storage service for the current scale.

# 6 Work Completed

## 6.1 Research and Design

- Conducted a requirements analysis with mess administration and students to identify key pain points (long queues, proxy entry, food wastage, lack of feedback).
- Evaluated authentication options and selected Microsoft Azure AD SSO due to JKLU's existing Microsoft 365 infrastructure.
- Designed the database schema with eleven Mongoose models spanning user management, meal tracking, menu configuration, payments, nutrition, polling, and complaints.
- Defined the REST API contract with eight route groups covering authentication, scanning, dashboard, administration, complaints, nutrition, payments, and polls.
- Designed UI wireframes for all nine application pages, incorporating modern design patterns such as glassmorphism, gradient-coded tiles, and blur-modal interactions.

- Selected the MERN stack based on team expertise and the requirement for a real-time, responsive single-page application.
- Evaluated payment gateway options and selected Razorpay for non-vegetarian meal bookings due to its comprehensive Node.js SDK and test mode support.

## 6.2 Backend

The complete backend has been implemented with the following components:

### 6.2.1 Server Setup (`server/index.js`)

- Express server with CORS configured for the React frontend at `localhost:5173`.
- JSON body parsing via `express.json()`.
- Static file serving from `uploads/` for complaint and review photos.
- MongoDB Atlas connection using `MONGO_URI` from environment variables.
- Eight route groups mounted under `/api/`: auth, mess, dashboard, admin, polls, nutrition, complaints, and payment.

### 6.2.2 Database Models (`server/models/`)

Eleven Mongoose models have been implemented:

| Model | Purpose |
|---|---|
| User | Student/admin profiles — name, email, rollNumber, role (5 roles), dietaryPreference, residencyStatus, foodAllergies, isBlocked |
| MealLog | Real-time scan entries — studentId, mealType, scannedAt timestamp |
| Menu | Weekly menu — dayOfWeek, mealType, veg items array, nonVegItems array |
| MealBooking | Advance meal skip/book decisions with compound unique index on {studentId, date, mealType} |
| Booking | Extended booking model with food item exclusion support |
| NonVegBooking | Paid non-veg orders — Razorpay fields (orderId, paymentId, signature), tiered pricing, status (pending/paid/failed) |
| Nutrition | Per-dish nutritional data (calories, protein, carbs, fat, fibre, sugar) for 100+ dishes |
| PollPost | Community issue posts with upvote/downvote arrays, category tags, two-step resolution |
| Complaint | Student complaints with text, image, text index on studentName, date index |
| HostellerRegistry | Hosteller email whitelist with cascading User model updates on login |
| Notice | Admin broadcast notices — message and date |

Table 2: Database Models (11 Collections)

#### 6.2.3 Controllers (`server/controllers/`)

**Authentication Controller (`authController.js`):**

- `microsoftLogin`: Receives {name, email, rollNumber} from the frontend. Validates the `@jklu.edu.in` domain (rejects with HTTP 403 otherwise). Uses the upsert pattern to find or create the user in MongoDB. Auto-detects Hosteller or Day-Scholar status via `HostellerRegistry` lookup on every login. Returns the user object with id, name, email, role, residencyStatus, and dietaryPreference.
- `getSettings / updateSettings`: Fetches and updates a student's editable profile fields — dietary preference and food allergies — persisted to MongoDB and returned to the frontend for `localStorage` synchronisation.

**Mess Controller (`messController.js`):**

- `scanQRCode`: Core entry verification. Validates student existence and block status.

Determines meal type via server clock. Checks duplicate entries. Creates a `MealLog` entry on success.

**Dashboard Controller (`dashboardController.js`):**

- `getDashboardData`: Fetches today's and tomorrow's menus (veg and non-veg), the full weekly menu, monthly skip statistics with remaining quotas, meal pricing, and tomorrow's existing bookings.
- `toggleMeal`: Handles meal skip/un-skip with server-side quota enforcement. Uses `findOneAndUpdate` with `upsert: true`.
- `getStudentHistory`: Returns all meal booking records for a student, sorted by most recent first.

**Admin Controller (`adminController.js`) — 10 functions:**

- `updateMenu`: Upserts veg and non-veg menu items for a given day and meal type.
- `getHeadcount`: Counts tomorrow's cancellations per meal type, cross-referenced with total hosteller count.
- `getRefundLedger`: Aggregates cancelled meals per student with user identity resolution for rebate processing.
- `updateNotice / getNotice`: CRUD for broadcast notices.
- `getAllStudents / toggleBlockStatus`: Student management with block/unblock functionality.
- `getHostellers / registerHosteller / deregisterHosteller`: Full CRUD for the Hosteller Registry whitelist with cascading User model updates.
- `updateUserRole`: Role assignment with validation against five valid roles.

**Payment Controller (`paymentController.js`):**

- `createOrder`: Creates Razorpay orders with tiered pricing (egg: Rs.30, chicken: Rs.120). Prevents duplicate bookings. Saves pending `NonVegBooking`.
- `verifyPayment`: HMAC-SHA256 signature verification using Node.js `crypto` module. Marks booking as paid or failed.
- `getBookingStatus`: Retrieves paid non-veg bookings for a student on a given date.
- `mockSuccess`: Development-only endpoint for testing without the Razorpay gateway.

**Poll Controller (`pollController.js`):**

- `getPosts`: Fetches active or resolved posts with category filter, regex search, and three sort algorithms (recent, top, trending with decay-weighted scoring).
- `createPost`: Hosteller-only post creation with category tagging.
- `vote`: Atomic upvote/downvote using `$addToSet` and `$pull`.
- `adminResolve / studentResolve`: Two-step resolution workflow.
- `deletePost`: Creator-only deletion.

### 6.2.4 Routes (`server/routes/`)

| Route Group | Method & Endpoint | Purpose |
|---|---|---|
| authRoutes | POST /api/auth/microsoft-login | Microsoft SSO login |
| | GET/PUT /api/auth/settings | Student settings |
| messRoutes | POST /api/mess/scan | QR scan verification |
| dashboardRoutes | GET /api/dashboard/data | Dashboard data |
| | POST /api/dashboard/toggle | Toggle meal skip |
| | GET /api/dashboard/history | Meal history |
| adminRoutes | POST /api/admin/menu | Update menu |
| | GET /api/admin/headcount | Headcount analytics |
| | GET /api/admin/ledger | Refund ledger |
| | POST/GET /api/admin/notice | Notices |
| | GET/POST/DELETE /api/admin/hostellers | Hosteller registry |
| | POST /api/admin/update-role | Role management |
| complaintRoutes | POST /api/complaints | Submit complaint |
| | GET /api/complaints | Search complaints |
| nutritionRoutes | GET /api/nutrition | All nutrition data |
| | GET /api/nutrition/search | Fuzzy search |
| | POST /api/nutrition | Add custom dish |
| paymentRoutes | POST /api/payment/create-order | Razorpay order |
| | POST /api/payment/verify | Verify payment |
| | GET /api/payment/status | Booking status |
| pollRoutes | GET/POST /api/polls | List/create posts |
| | POST /api/polls/:id/vote | Vote on post |
| | PUT /api/polls/:id/*-resolve | Resolution |

Table 3: Complete API Route Summary (8 Groups, 26+ Endpoints)

### 6.2.5 Database Seeder and Utility Scripts

- `seedMenu.js`: Populates MongoDB with sample weekly menu data including veg and non-veg items.
- `fixRole.js`: Administrative script for correcting user role inconsistencies.
- `mess_nutrition.json`: Seed data containing nutritional information for 100+ Indian mess dishes.

## 6.3 Frontend

The complete frontend has been implemented with nine page components and four reusable components:

### 6.3.1 Application Setup

- **main.jsx**: Creates an MSAL instance using Azure AD credentials from environment variables (`VITE_CLIENT_ID`, `VITE_AUTHORITY`). Wraps the entire app in `<MsalProvider>`.
- **App.jsx**: Defines React Router with nine protected routes — `/` (AuthGate), `/dashboard`, `/menu`, `/scanner`, `/admin`, `/history`, `/settings`, `/student-management`, and `/voting`.

### 6.3.2 Reusable Components

- **PrivateRoute.jsx**: Auth guard that checks `localStorage` and redirects unauthenticated users.
- **Navbar.jsx**: Responsive navigation bar with role-based links (Home, Menu, History, Settings, Staff Panel, Voting), user name display, and logout.
- **NonVegBookingModal.jsx**: Full Razorpay checkout flow — dynamically loads the Razorpay script, displays tiered pricing, handles payment lifecycle (details, paying, success, error), and includes a mock payment mode for development.
- **DishSearchInput.jsx**: Fuzzy search autocomplete with debounced API calls (280ms), tag-based selection, and an inline custom dish creation form for adding new dishes to the Nutrition database.

### 6.3.3 Pages

**AuthGate.jsx — Login Page:**

- Full-screen background with JKLU mess imagery and glassmorphism login card.
- Microsoft SSO popup login via MSAL.
- Frontend email domain validation (`@jklu.edu.in`).
- Stores user session (including role, residencyStatus, dietaryPreference) in `localStorage`.

**Dashboard.jsx — Student Dashboard:**

- Animated notice banner with shimmer animation and pulsing badge.

- Digital Mess Pass (ID card) with QR code and gradient-bordered hover glow effect.
- Monthly skip statistics showing remaining quota per meal type with meal pricing display.
- Today's and tomorrow's menus as colour-coded gradient meal tiles with click-to-expand blur-modal popups showing dish lists and nutritional information.
- Tomorrow's menu with skip/book toggles, food item exclusion, and non-veg purchase via Razorpay modal.
- Complaint submission form with text input and optional photo upload.
- Review and star-rating form for each meal with photo evidence upload.
- Full weekly menu grid with day-wise expandable cards.

**MenuPage.jsx — Dedicated Menu Page:**

- Day-selector tabs for browsing the full weekly menu.
- Today's menu as four vibrant gradient tiles in a responsive grid.
- Click-to-expand blur-backdrop modal with per-dish nutritional information in expandable panels with animated progress bars.
- Dietary preference filtering (Vegetarian/Eggetarian/Jain hides or shows non-veg items accordingly).

**Scanner.jsx — Mess Staff QR Scanner:**

- Camera-based QR code scanning using `@yudiel/react-qr-scanner`.
- Validates student existence, block status, and duplicate entries.
- Visual feedback: green checkmark on success, red X on failure. Auto-reset after 3 seconds.

**AdminDashboard.jsx — Staff Portal:**

- Sidebar navigation with role-gated sections:
  - **Kitchen Controls** (admin, contractor): Menu management with `DishSearchInput` fuzzy autocomplete for veg/non-veg items. Live headcount analytics with total hosteller count.
  - **Refund Ledger** (admin, accountant): Accordion view of per-student cancelled meals with CSV export for rebate processing.

- **Complaints** (admin, contractor): Searchable complaint grid with date filtering and photo evidence.
- **Notice Broadcast** (admin): Campus-wide mess announcements.
- **Hosteller Registry** (admin): Register/deregister hostellers with cascading updates.
- **Role Management** (admin): Assign any of five roles to users.

**Settings.jsx — Student Profile:**

- Three-section card layout: Personal Information (read-only from OAuth), Residency & Diet (editable dietary preference with visual chips), Food Allergies (free-text).
- Dietary changes immediately update menu filtering via `localStorage` synchronisation.

**Voting.jsx — Community Issue Polling:**

- Reddit-style two-panel layout with inline vote buttons and detail panel with SVG donut chart.
- Sort tabs (Recent, Trending, Top), category filter chips, debounced search (300ms).
- Active/Archive toggle with two-step resolution workflow.
- Hosteller-only post creation with category tagging.

**StudentManagement.jsx — Admin Student Management:**

- Searchable student list with block/unblock toggles and role assignment dropdowns.

**History.jsx — Cancellation History:**

- Tabular display of past meal cancellations with date, meal type, and status.

## 6.4 UI/UX Design

The frontend employs a cohesive, premium design system built with vanilla CSS3:

- **Glassmorphism**: Frosted-glass effects using `backdrop-filter: blur(10px) saturate(1.4)` for modal backdrops.
- **Gradient Meal Tiles**: Colour-coded interactive tiles with distinct gradients per meal type, glow orbs, and spring-physics hover animations (`cubic-bezier(.34,1.56,.64,1)`).

- **Micro-Animations**: Shimmer effects, pulsing badges, bell ring animations, and smooth modal slide-up entry (`@keyframes modalSlideUp`).
- **Responsive Design**: CSS Grid and Flexbox with media queries for 480px and 768px breakpoints.
- **Warm Accent Theme**: Amber/orange palette, card-based layouts with subtle shadows, and clear visual hierarchy.
- **SVG Data Visualisation**: Custom donut charts in the polling system rendered via inline SVG with computed stroke-dasharray values.

# 7 Risks and Mitigation Strategies

| S.No. | Risk | Mitigation Applied |
|---|---|---|
| 1 | **QR Code Sharing/Fraud**: Students may share screenshots of their QR code with others for proxy entry. | Implemented server-side duplicate detection preventing the same QR from being scanned twice for the same meal on the same day. Added student block/unblock functionality for administrative enforcement. |
| 2 | **Network Dependency**: The QR scanner requires an active internet connection. Network outages at the mess entrance would block entries. | The QR code is generated client-side and cached in the student dashboard. A manual student ID entry fallback is available in the mess staff interface. |
| 3 | **Camera/Device Compatibility**: The web-based QR scanner may not work reliably across all devices and browsers. | Tested across multiple devices and browsers. The `@yudiel/react-qr-scanner` library provides broad compatibility with graceful degradation. |
| 4 | **Payment Security**: Non-vegetarian meal payments could be forged or manipulated. | Implemented HMAC-SHA256 cryptographic signature verification for all Razorpay payments. Payment status is verified server-side before confirming bookings. Idempotent duplicate prevention at the database level. |

| S.No. | Risk | Mitigation Applied |
|---|---|---|
| 5 | **Data Consistency**: Race conditions during concurrent meal toggles or scans could lead to duplicate or inconsistent records. | Leveraged MongoDB's atomic `findOneAndUpdate` operations and unique compound indexes on `{studentId, date, mealType}` to prevent duplicates at the database level. |
| 6 | **Scalability**: As the user base grows, the current architecture may face performance bottlenecks. | Added database indexes on frequently queried fields (text index on Complaint, date-sorted indexes). Implemented API response capping (fuzzy search limited to 10 results). Architecture supports horizontal scaling. |
| 7 | **Unauthorized Access**: Users might attempt to access admin-only features. | Implemented five-role RBAC (student, admin, contractor, accountant, controller) with both frontend conditional rendering and backend role validation. PrivateRoute guards protect all authenticated pages. |

Table 4: Risk Assessment and Mitigation

# 8 Conclusion

The JKLU Mess Management Portal has been successfully developed as a comprehensive, production-ready web application that addresses all identified pain points in the existing mess management system. The project delivers a complete digital transformation of the mess entry, menu management, and feedback ecosystem at JK Lakshmipat University.

The key accomplishments of this project include:

1. **Digital Entry Verification**: Replaced physical ID card checking with QR code-based scanning, eliminating proxy entries and enabling real-time meal logging with time-based meal detection.

2. **Advance Meal Management**: Implemented a quota-based meal skip/booking system with food item exclusion, enabling students to customise their meals in advance while providing the kitchen with accurate headcount data to reduce food wastage.

3. **Payment Integration**: Integrated Razorpay payment gateway for non-vegetarian meal bookings with HMAC-SHA256 cryptographic verification, supporting tiered pricing for egg and chicken items.

4. **Nutritional Awareness**: Built a comprehensive database of 100+ dishes with detailed nutritional information (calories, protein, carbohydrates, fat, fibre, sugar), accessible via fuzzy search and displayed alongside every menu item.

5. **Community Feedback**: Developed a Reddit-style polling system with upvote/-downvote mechanics, trending algorithms, and a two-step resolution workflow, giving students a structured voice in mess improvement.

6. **Administrative Tools**: Provided mess administrators with kitchen headcount analytics, refund ledger with CSV export, hosteller registry management, student block/unblock controls, role management, complaint tracking, and notice broadcasting capabilities.

7. **Premium User Experience**: Delivered a modern, responsive UI featuring glassmorphism, gradient-coded meal tiles, blur-modal interactions, micro-animations, and dietary preference-aware filtering — creating an engaging, app-like experience.

The final system comprises eleven database models, six backend controllers, eight RESTful API route groups with over 26 endpoints, nine frontend page components, and four reusable UI components — built on the MERN stack with Microsoft Azure AD SSO and Razorpay payment integration. The application demonstrates the practical application of modern web development concepts including OAuth 2.0 authentication, cryptographic payment verification, NoSQL database design with compound indexes, atomic database operations, RESTful API architecture, and responsive CSS3 design patterns.

# 9 Appendix

## A.1: Project Screenshots

The following figures illustrate key screens of the JKLU Mess Management Portal.

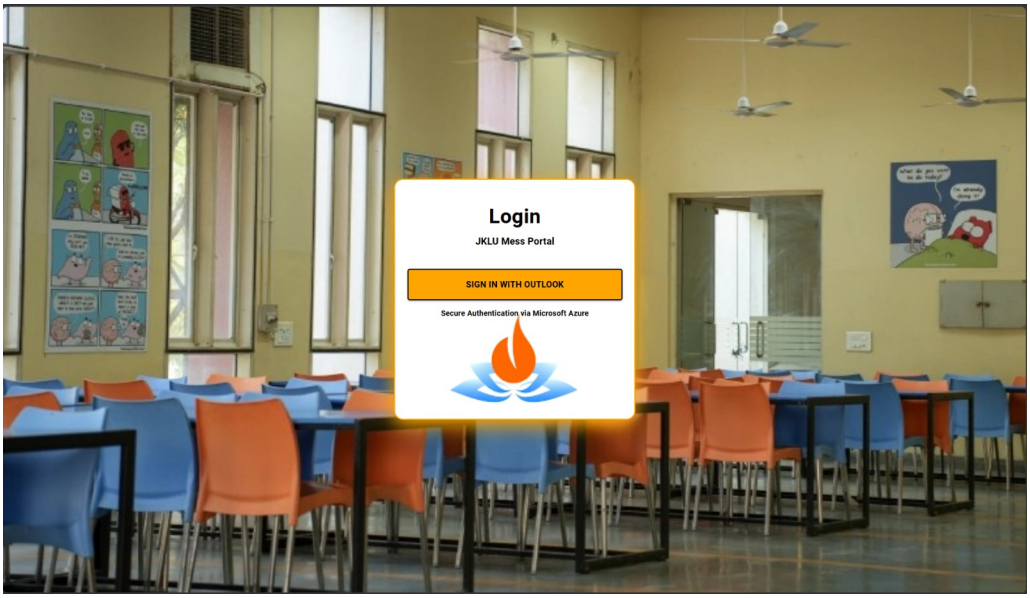


Figure 2: Login Interface

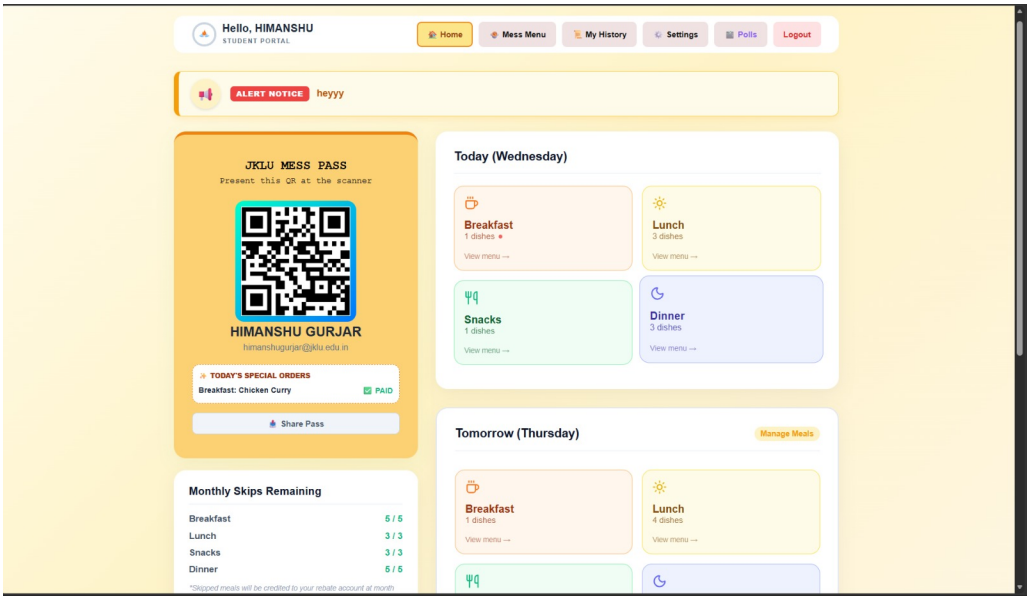


Figure 3: Student Dashboard — QR Code Mess Pass and Meal Overview

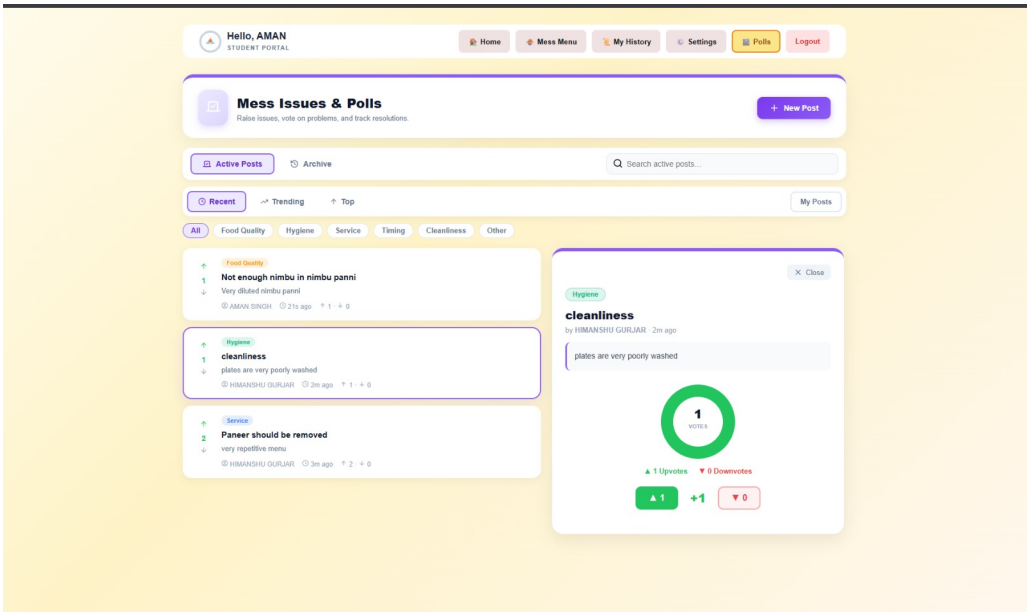


Figure 4: Poll Voting System

## A.2: References


1. MongoDB Documentation. *MongoDB Manual*. https://www.mongodb.com/docs/manual/
2. Express.js. *Express – Node.js Web Application Framework*. https://expressjs.com/
3. React. *React Documentation*. https://react.dev/
4. Microsoft Identity Platform. *Microsoft Authentication Library (MSAL) for JavaScript*. https://learn.microsoft.com/en-us/entra/identity-platform/msal-overview
5. Mongoose ODM. *Mongoose v8.x Documentation*. https://mongoosejs.com/docs/
6. Vite. *Vite – Next Generation Frontend Tooling*. https://vitejs.dev/
7. Razorpay. *Razorpay Payment Gateway Documentation*. https://razorpay.com/docs/
8. QR Server API. *QR Code Generator API*. https://goqr.me/api/
9. @yudiel/react-qr-scanner. *React QR Scanner Component*. https://www.npmjs.com/package/@yudiel/react-qr-scanner
10. OAuth 2.0 Authorization Framework. *RFC 6749*. https://datatracker.ietf.org/doc/html/rfc6749

11. Fielding, R.T. (2000). *Architectural Styles and the Design of Network-based Software Architectures*. Doctoral dissertation, University of California, Irvine.

12. Multer. *Node.js Middleware for Handling multipart/form-data*. https://www.npmjs.com/package/multer

13. Node.js Crypto Module. *Node.js Crypto Documentation*. https://nodejs.org/api/crypto.html

14. CSS Backdrop Filter. *MDN Web Docs — backdrop-filter*. https://developer.mozilla.org/en-US/docs/Web/CSS/backdrop-filter